Operációs rendszer: nincs általános definíciója; köztes réteg a hardver és a felhasználó között; feladata a felhasználói programok futtatása és a sz.gép használatának egyszerűvé tétele, valamint az erőforrások kezelése és elosztása és a versenyhelyzetekben való döntés maximális hatékonysággal.

Felépítése: lehet
- *egyszerű struktúrájú*: (~MS-DOS, eredeti Unix) limitált tudású rendszer, lehető legkisebb helyfoglalással; nem különülnek el igazán a rendszerhívások a funkcionális rétegtől; nehéz implementáció/karbantartás
- *réteges felépítésű*: 0.réteg a hardver, a legfelső pedig a felhasználói felület. Egy réteg különböző adatstruktúrákból épül fel, illetve olyan függvényekből, amikkel felsőbb rétegekből kezelni lehet az adatokat – minden réteg az alatta levő eljárásait képes használni, így egyszerű a tervezés/hibakeresés, viszont nem túl hatékony, hogy egy hívás sok szükségtelen rétegen is áthaladhat.

Rendszermag (kernel): OS alapja, hardver erőforráskezeléséért felel. Típusai:
- Mikrokernel: csak a legszükségesebb alkotóelemekből áll, minden más rendszer- és felhasználói programokkal kerül bele. Minimális memória- és folyamatkezelést szolgáltat, és némi kommunikációs képességet. Egyszerűen bővíthető.
- Monolitikus kernel: egyetlen megbonthatatlan egységet képző modul, nagyteljesítményű, de nehéz bővíteni (újrafordítást igényel).
Monolitikus kernelnél a kernel modulok egy úgynevezett kerneltérben futnak, ahol elérik a kernel globális változóit, míg a mikrokernel esetén van egy mag és a mag köré épülnek a modulok, amik megadott szabványokon keresztül cserélnek információt egymással. A mikro- és monolitikus közötti megoldás a hibrid kernel.
Exokernel: csak egy programokban használható rutinkönyvtárból áll, a programozó döntése, hogy mely részeket kívánja használni.

Folyamat: általában egy futó program (=kód + aktuális aktivitás jellemzői (programszámláló+regiszterállapotok), stack az ideiglenes adatok tárolására, adatszegmens a globális változóknak, heap a dinamikusan foglalt memóriának). Program≠folyamat, csak ha már betöltődött a memóriába. Egy folyamat a végrehajtása során különböző állapotok vesz fel, amit a folyamat aktuális tevékenysége határoz meg. Öt állapot lehetséges:
- *Új*: az új folyamat létrehozás alatt áll
- *Futó*: a program utasításai végrehajtás alatt állnak (a program fut) (egy CPU-n egy időpillanatban csak 1 ilyen lehet)
- *Várakozó*: a folyamat egy esemény bekövetkezésére vár (pl.: I/O műveletre)
- *Készenléti*: a folyamat várja, hogy processzoridőhöz jusson
- *Véget ért*: a folyamat befejezte a végrehajtást

Két folyamat egymással két féle kapcsolatban állhat: vagy *független*ek egymástól, ekkor semmilyen módon nem tudják egymás állapotát befolyásolni, vagy pedig *együttműköd*nek, ekkor viszont befolyásolhatják egymás állapotát.
Folyamatközi kommunikációra két jellemző megoldás van, az egyik az *osztott memória alapú* megoldás (gyors, kényelmes), a másik az *üzenet alapú kommunikáció* (egyszerűbb implementálni).
Direkt kommunikáció esetén az üzeneteknél címzett és/vagy feladó is meg van jelölve, i*ndirekt*nél postafiókon keresztül történik a kézbesítés.
A kommunikáció lehet: *szinkron vagy aszinkron*. Ha a kommunikációs csatornának nincs kapacitása (egy folyamat egy üzenet elküldése után addig nem küldhet új üzenetet, amíg az előző célba nem ért), akkor a kapcsolat szinkron, ha van kapacitása (csatorna csak akkor blokkolódik, ha elfogy a kapacitás), akkor a kapcsolat aszinkron.

Folyamatütemező: eldönti, hogy éppen melyik folyamatot futtassa a CPU. Megléte szükséges az erőforrások lehető leghatékonyabb kihasználása érdekében, és a programok folyamatos futásának érzetéhez.
Ütemezési sorok:
- job queue (feladatsor): ide kerülnek az új folyamatok.
- ready queue (készenléti sor): memóriába betöltött, futásra kész folyamatok kerülnek bele
+ egyéb sorok, pl. eszközvárakozási sor: I/O eszközök használatára váró folyamatok (eszközönként saját)
Az ütemező e sorokból választja ki a folyamatokat. Ütemezők két jellemző típusa:

- Hosszú távú ütemező (Long-term scheduler, HTÜ): ez választja ki a feladatsorból a folyamatokat, és teszi át őket a memóriába végrehajtásra
- Rövid távú ütemező (Short-term schedulerm, RTÜ): ez dönti el, hogy a ready queue elemeiből éppen melyik folyamat használhassa a CPU-t. (A kettő közül ez fut le gyakrabban.)

Az ütemező célja a CPU lehető legnagyobb mértékű terhelése, és az adott idő alatt elvégezhető munkaegységek számának maximalizálása. Figyelembe veendő: a folyamatok lefutási ideje, várakozási ideje (ready queue-ban), és a válaszidejük.

Nem preemptív (kooperatív) ütemezés: akkor beszélünk erről, ha két feltétel teljesülése esetén van szükség CPU ütemezésre: amikor egy folyamat kilép, vagy futó állapotúból várakozó folyamatba vált. Tehát a CPU nem veheti el mindaddig a CPUhasználat jogát. (**Preemptív**nél elveheti.)

First Come First Serve (FCFS) algoritmus: a folyamatok érkezési sorrendben hajtódnak végre. Hosszúak a várakozási idők, „konvojeffektus" alakulhat ki. A kihasználtság itt alacsonyabb (CPU / I/O téren)

Prioritásos algoritmusok: folyamatokhoz prioritást rendelnek és eszerinti sorban hajtódnak végre. Megegyező prioritásúaknál FCFS algoritmus szerint. Legnagyobb problémája a *kiéheztetés*, amikor egy alacsony prioritású folyamat sosem kerül sorra, mert mindig van előtte egy magasabb.

Shortest Job First algoritmus: a várakozási sorból mindig a legkisebb CPU bursttel rendelkező folyamat fut le.
(CPU burst (löketidő): milyen hosszan használja egy folyamat a CPU-t az I/O várakozások között.)

Round-robin algoritmus: időosztásos rendszerekre fejlesztették ki. A ready queue egy FIFO sor, amin megy végig az ütemező, és folyamatonként ad 1-1 CPU időszeletnyi időt. Ha a processz az időszelet lejárta előtt végez, a CPU felszabadítja magát, és az ütemező átadja a köv. folyamatot. Ha a lejártakor nem végez, megszakítás történik, és a folyamat a ready queue végére kerül.

Lottery scheduling: Minden processz ticketeket kap, amelynek több a ticketje, nagyobb eséllyel fog futási jogot kapni. A kooperáló processzek ticketet adhatnak át egymásnak.

Fair-share scheduling: Ha egy felhasználó több processzt futtat, mint a többi, akkor több processzoridőt kap, hiszen nagyobb az esélye annak, hogy az ő processzei kapnak futási jogot.

Többszintű ütemezés: akkor használható jól, ha a folyamatok egyszerűen sorolhatók csoportokba. Jellemzően az előtérben és háttérben futó programokat különböztetik meg, mert ezek különböző ütemezési- és válaszidő-elvárásokkal rendelkeznek. A ready queue itt több elválasztott sorra oszlik fel, ahol mindnek más az ütemező algoritmusa. Egy folyamat csak egy ilyen sorban lehet, és mindig valamilyen tulajdonság alapján kerül besorolásra. A sorok közt prioritásos sorrend áll fenn.

Többszintű, visszacsatolásos ütemezés: Itt egy folyamat már mozoghat az egyes sorok között. A folyamatok CPU felhasználás alapján sorolódnak be egy-egy sorba. Ha egy folyamat az adott sorhoz rendelt határnál több CPU időt használna fel, akkor bekerül egy kisebb prioritású sorba. Ha egy folyamat túl hosszú ideje lenne egy sorban, akkor az ütemező áthelyezi egy magasabb prioritású sorba, így kerülve el a kiéheztetést.

Memóriakezelés alapelvei:

Fizikai és logikai memória: azon memóriacímeket, amelyeket a CPU generál, logikai címeknek nevezzük, míg amiket a memóriakezelő egység lát (amik a memória memóriacím regiszterébe kerülnek), fizikai címeknek hívjuk.

MMU (memóriamenedzsment-egység): ez végzi a futásidőben történő virtuális => fizikai címátalakítást.

Virtuális memória: OS vagy a hardver által nyújtott szolgáltatás, a merevlemez igénybevételével, a futó program(ok) számára transzparens módon biztosítják, hogy a program végrehajtáskor a központi vagy operatív memória fizikai korlátai észrevétlenek legyenek. Az operációs rendszer úgy szabadít fel operatív memóriát az éppen futó program számára, hogy a memóriában tárolt, de éppen nem használt blokkokat (lapokat) kiírja a külső tárolóra, amikor pedig ismét szükség van rájuk, visszaolvassa őket (=> swapping).

Lapozás (paging): memóriakezelési technika, ami megengedi, hogy egy folyamat fizikai címtartománya ne legyen folyamatos. Ez megoldja a változó méretű memóriaszeletek háttértárra helyezésének problémáját is. A fizikai memória fixméretű blokkokra (frame-ek) osztódik, és a logikai memória is (lapok).

A memóriának két fajtája vank, a ROM típusú csak olvasható (van újraírható: EPROM), a RAM pedig írható + olvasható.

Gyorsítótár (cache): átmeneti információtároló elemek, melyek célja az információ-hozzáférés gyorsítása. A gyorsítás azon alapul, hogy a gyorsítótár gyorsabb tárolóelem, mint a hozzá kapcsolt, gyorsítandó működésű elemek.
Teljesen asszociatív cache esetén a gyorsítandó memóriaegység beolvasott blokkjának tartalma átkerülhet a gyorsítótár bármelyik blokkjának területére, míg *közvetlen (direkt) leképzésű cache* esetén a blokk a cache meghatározott blokkjára kerülhet csak.